DOSSIER DE CONCEPTION TECHNIQUE

Shared Documents Editor

Plateforme collaborative d'édition et de partage de documents

Architecture Cloud AWS

Infrastructure as Code, collaboration temps réel, sécurité et scalabilité

Auteur :	Samuel Sean FOTSING
Technologies principales :	TypeScript, React, AWS SAM
	Lambda, API Gateway, DynamoDB, S3
Région AWS envisagée :	eu-north-1 (Stockholm)
Version du document :	1.0
Date :	26 octobre 2026

Table des matières

1 Présentation générale **4**

1.1 Contexte 4

1.2 Problématique 4

1.3 Objectifs 4

1.4 Périmètre fonctionnel 5

1.5 Hypothèses et limites 5

2 Analyse des besoins **6**

2.1 Acteurs 6

2.2 Besoins fonctionnels 6

2.3 Besoins non fonctionnels 7

2.4 Règles de gestion 7

3 Architecture générale **9**

3.1 Choix architectural 9

3.2 Vue d'ensemble 9

3.3 Rôle des composants 9

3.3.1 Frontend React et TypeScript 9

3.3.2 AWS Lambda 10

3.3.3 Amazon API Gateway 10

3.3.4 Amazon DynamoDB 10

3.3.5 Amazon S3 10

3.3.6 AWS SAM et CloudFormation 10

3.3.7 Amazon CloudWatch 11

3.3.8 Amazon SES 11

4 Conception de la collaboration temps réel **12**

4.1 Problème de la concurrence 12

4.2 Utilisation de Yjs 12

4.3 Protocole de synchronisation 12

4.4 Présence et curseurs 13

4.5 Persistance et reconnexion . . . 13
4.6 Versions et restauration . . . 13

5 Modèle de données 14
5.1 Principes . . . 14
5.2 Entités principales . . . 14
5.3 Clés et index . . . 15
5.4 Persistance des contenus . . . 15

6 Gestion des quotas et téléchargements 16
6.1 Quota quotidien de rédaction . . . 16
6.1.1 Contrôle atomique . . . 16
6.2 Création d'un lien de partage . . . 16
6.3 Contrôle concurrent des téléchargements . . . 17
6.4 Réservation et livraison effective . . . 17
6.5 Livraison des PDF . . . 17
6.6 Notification du dernier téléchargement . . . 18

7 Sécurité 19
7.1 Gestion des identités et des permissions . . . 19
7.2 Protection des données . . . 19
7.3 Protection contre les abus . . . 19
7.4 Sécurité réseau . . . 20
7.5 Gestion des secrets . . . 20

8 Scalabilité et disponibilité 21
8.1 Hypothèse de charge . . . 21
8.2 Stratégie de montée en charge . . . 21
8.2.1 Lambda . . . 21
8.2.2 API Gateway . . . 21
8.2.3 DynamoDB . . . 22
8.2.4 S3 . . . 22
8.3 Disponibilité et reprise . . . 22

9 Infrastructure as Code et déploiement 23
9.1 Rôle d'AWS SAM . . . 23
9.2 Organisation du dépôt . . . 23
9.3 Étapes de validation . . . 24
9.4 Commandes indicatives . . . 24
9.5 Suppression des ressources . . . 24

10 Plan de tests **25**
10.1 Objectifs . . . 25
10.2 Tests unitaires . . . 25
10.3 Tests d'intégration . . . 25
10.4 Tests de concurrence . . . 25
10.5 Tests de charge . . . 26
10.6 Métriques à recueillir . . . 26
10.7 Rapport de test . . . 27

11 Documentation API **28**
11.1 Principes . . . 28
11.2 Routes HTTP envisagées . . . 28
11.3 Codes de réponse . . . 28

12 Coûts et maîtrise budgétaire **30**
12.1 Principaux postes de coût . . . 30
12.2 Méthode d'estimation . . . 30
12.3 Alertes et limites . . . 30

13 Risques et mesures d'atténuation **32**

14 Plan de réalisation **33**
14.1 Critères de validation . . . 33

15 Conclusion **35**

Chapitre 1

Présentation générale

1.1 Contexte

Shared Documents Editor est une application Web permettant à plusieurs utilisateurs de créer, modifier et partager des documents textuels de manière collaborative.

L'application vise à offrir une expérience simple de coédition, tout en garantissant la cohérence des documents, la sécurité des données et le contrôle du partage.

Elle repose sur une architecture cloud utilisant les services AWS. L'infrastructure est décrite sous forme de code afin de rendre les déploiements reproductibles et de faciliter la maintenance.

1.2 Problématique

Les principales difficultés à résoudre sont les suivantes :

— permettre à plusieurs utilisateurs de modifier simultanément un même document ;
— éviter la perte de modifications concurrentes ;
— conserver un historique des versions et permettre la restauration d'une version antérieure ;
— générer et partager des PDF à l'aide de liens soumis à une limite de téléchargements ;
— faire respecter un quota quotidien de caractères pour les rédacteurs ;
— détecter et limiter les comportements abusifs ;
— supporter une arrivée importante de visiteurs ;
— superviser les performances, les erreurs et les coûts.

1.3 Objectifs

L'objectif général est de concevoir et réaliser une plateforme collaborative fiable, sécurisée, évolutive et déployable sur AWS.

Les objectifs spécifiques sont :

1. proposer une interface Web responsive ;
2. synchroniser les modifications des documents en temps réel ;
3. persister les documents et leur historique ;
4. contrôler atomiquement les quotas et les téléchargements ;
5. stocker les fichiers dans un espace privé ;
6. appliquer le principe du moindre privilège ;

7. automatiser la création de l'infrastructure ;
8. mesurer les performances par des tests reproductibles ;
9. documenter les procédures de déploiement et de suppression.

1.4 Périmètre fonctionnel

Le système couvre les fonctionnalités suivantes :

— création et édition de documents ;
— partage d'un document par lien ;
— présence des collaborateurs et affichage des curseurs ;
— sauvegarde et consultation des versions ;
— restauration d'une version ;
— génération de PDF ;
— création de liens de téléchargement limités ;
— application d'un quota de rédaction de 10 000 caractères par jour ;
— blocage temporaire en cas d'abus ;
— tableau de bord pour les associations ;
— notification par e-mail à l'épuisement du quota d'un lien, selon les règles définies.

1.5 Hypothèses et limites

Les hypothèses de conception sont les suivantes :

— le frontend est une application React avec TypeScript ;
— le backend utilise Node.js et TypeScript sur AWS Lambda ;
— les API HTTP et WebSocket sont exposées par API Gateway ;
— DynamoDB assure le stockage des métadonnées et des états nécessitant des opérations atomiques ;
— S3 stocke les PDF et les objets documentaires persistants ;
— AWS SAM et CloudFormation décrivent l'infrastructure ;
— CloudWatch assure la supervision ;
— Amazon SES peut être utilisé pour les notifications.

Ces choix constituent l'architecture cible. Leur présence dans ce document ne signifie pas que toutes les ressources sont déjà déployées ou que toutes les fonctionnalités sont réalisées.

Les tailles maximales des documents, les limites de débit, la durée de conservation et les quotas de service devront être définis et vérifiés avant la mise en production.

Chapitre 2

Analyse des besoins

2.1 Acteurs

Acteur	Responsabilités
Visiteur	Accéder à un document partagé et télécharger un PDF si le lien est valide et disponible.
Rédacteur	Créer et modifier des documents, inviter des collaborateurs et gérer les versions.
Collaborateur	Participer à la modification d'un document partagé, dans les limites des permissions accordées.
Gestionnaire d'association	Consulter les documents et les indicateurs accessibles à son association.
Service de notification	Envoyer les e-mails déclenchés par les événements métier.
Administrateur technique	Superviser l'application et intervenir sur l'infrastructure selon les autorisations prévues.

2.2 Besoins fonctionnels

ID	Fonctionnalité	Description
BF-01	Créer un document	Créer un document et lui attribuer un identifiant unique.
BF-02	Modifier un document	Permettre la modification du contenu par un ou plusieurs rédacteurs autorisés.
BF-03	Synchroniser	Diffuser les modifications et les informations de présence aux collaborateurs connectés.
BF-04	Sauvegarder	Persister l'état cohérent du document.
BF-05	Historique	Enregistrer et consulter les versions.
BF-06	Restaurer	Restaurer une version en créant une nouvelle version courante, sans supprimer l'historique.
BF-07	Partager	Générer un lien doté de permissions définies.

BF-08	Exporter	Produire un PDF cohérent correspondant à une version déterminée du document.
BF-09	Télécharger	Contrôler le nombre de téléchargements autorisés par lien.
BF-10	Quota de rédaction	Limiter un rédacteur à 10 000 caractères par jour.
BF-11	Anti-abus	Détecter les comportements anormaux et appliquer des blocages temporaires.
BF-12	Tableau de bord	Afficher les informations autorisées concernant une association.
BF-13	Notification	Informer le créateur lorsqu'un lien a consommé son dernier téléchargement autorisé.
BF-14	Audit	Conserver les événements importants pour le diagnostic et la traçabilité.

2.3 Besoins non fonctionnels

ID	Exigence
BNF-01	Disponibilité : limiter les points uniques de défaillance et exploiter les mécanismes de résilience des services AWS.
BNF-02	Performance : limiter la latence des opérations courantes et mesurer les temps de réponse.
BNF-03	Concurrence : ne pas perdre les modifications valides et empêcher le dépassement des quotas critiques.
BNF-04	Sécurité : appliquer le moindre privilège, le chiffrement et la validation des entrées.
BNF-05	Scalabilité : adapter les capacités aux variations de charge et surveiller les limites de service.
BNF-06	Maintenabilité : séparer le frontend, le backend, l'infrastructure et les tests.
BNF-07	Reproductibilité : déployer les ressources à partir d'une définition d'infrastructure versionnée.
BNF-08	Traçabilité : produire des journaux exploitables sans exposer les contenus sensibles ou les secrets.
BNF-09	Maîtrise des coûts : mesurer les consommations et configurer des alertes budgétaires.

2.4 Règles de gestion

1. Chaque document possède un identifiant unique.
2. Un document ne peut être modifié que par un utilisateur disposant des permissions nécessaires.
3. Les modifications concurrentes doivent être fusionnées ou rejetées explicitement ; elles ne doivent pas écraser silencieusement des modifications valides.

4. Une restauration crée une nouvelle version à partir d'une version antérieure.
5. Le quota de rédaction est de 10 000 caractères par rédacteur et par jour, selon une définition uniforme du caractère.
6. Le compteur d'un lien ne peut pas devenir négatif.
7. Les liens invalides, expirés ou épuisés ne permettent plus de démarrer un nouveau téléchargement.
8. Le PDF délivré doit correspondre à une version cohérente et identifiée du document.
9. Un blocage temporaire possède une date d'expiration.
10. Les permissions sont vérifiées côté serveur ; masquer une fonctionnalité dans le frontend ne constitue pas une mesure de sécurité.

Chapitre 3

Architecture générale

3.1 Choix architectural

L'application adopte une architecture serverless sur AWS. Les services managés évitent d'administrer directement les serveurs d'application, mais nécessitent de comprendre les limites de débit, la concurrence, les permissions et les coûts.

L'infrastructure est décrite par AWS SAM, qui s'appuie sur AWS CloudFormation pour créer et gérer les ressources.

3.2 Vue d'ensemble

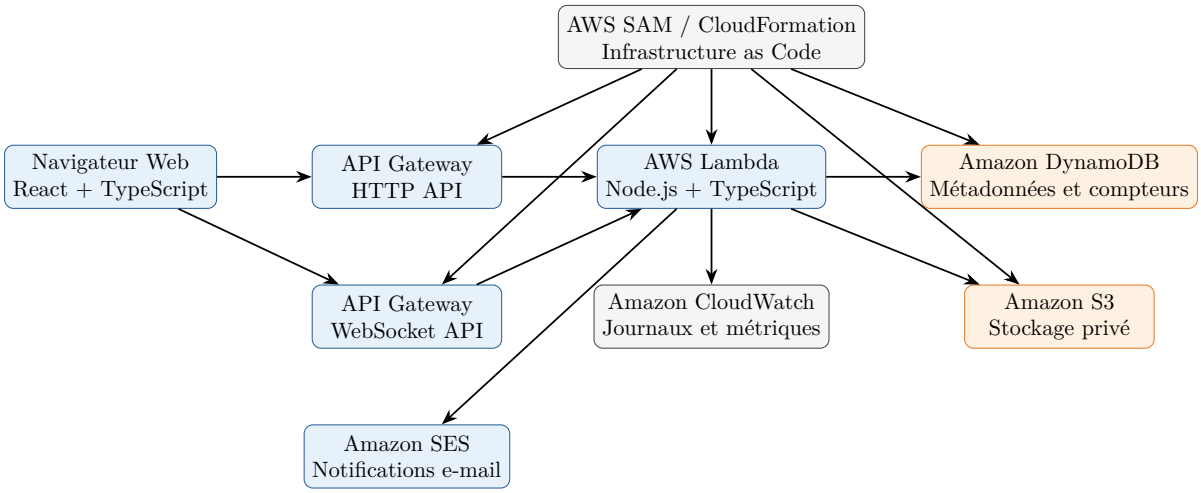


FIGURE 3.1 – Architecture logique cible de Shared Documents Editor

3.3 Rôle des composants

3.3.1 Frontend React et TypeScript

Le frontend fournit l'interface utilisateur. Il gère l'éditeur, les curseurs, la présence, les états de chargement, les erreurs et les interactions de partage.

Il communique avec le backend par HTTP pour les opérations classiques et par WebSocket pour les échanges en temps réel.

Le frontend ne détient aucune clé secrète AWS et ne décide jamais seul si un utilisateur possède une permission.

3.3.2 AWS Lambda

Lambda exécute les fonctions backend à la demande. Les fonctions peuvent traiter les requêtes HTTP, la gestion des liens, les opérations documentaires et les événements de collaboration.

Les fonctions doivent être conçues pour être idempotentes lorsque cela est nécessaire, gérer les erreurs et respecter les limites de durée d'exécution et de concurrence du service.

3.3.3 Amazon API Gateway

L'API HTTP expose les opérations métier telles que la création de documents, l'accès à l'historique et la génération de liens.

L'API WebSocket permet de maintenir les connexions des collaborateurs et de diffuser les messages de synchronisation.

Le protocole de transport WebSocket doit être compatible avec le protocole applicatif retenu. L'utilisation de Yjs ne rend pas automatiquement API Gateway compatible avec tous les serveurs WebSocket existants : un adaptateur applicatif est nécessaire.

3.3.4 Amazon DynamoDB

DynamoDB conserve les métadonnées, les permissions, les versions, les quotas, les compteurs de téléchargement et les états de blocage.

Les écritures conditionnelles permettent de protéger les invariants métier contre les requêtes concurrentes.

Les opérations critiques peuvent utiliser des transactions DynamoDB lorsque plusieurs éléments doivent être modifiés ensemble.

3.3.5 Amazon S3

S3 stocke les PDF et les objets persistants associés aux documents. Le bucket reste privé et l'accès passe par des rôles IAM et le backend autorisé.

Les clés d'objets sont déterministes ou versionnées afin de permettre d'identifier précisément le contenu stocké.

Les politiques de conservation et de suppression doivent être définies explicitement.

3.3.6 AWS SAM et CloudFormation

AWS SAM fournit une syntaxe simplifiée pour déclarer les ressources serverless. CloudFormation assure la gestion déclarative de ces ressources.

Le template constitue la référence de l'infrastructure : fonctions, API, tables, bucket, permissions et paramètres.

Les modifications doivent être versionnées et validées avant le déploiement.

3.3.7 Amazon CloudWatch

CloudWatch centralise les journaux et les métriques. Il permet de suivre les erreurs, la latence, la consommation des fonctions et les limites de capacité.

Les journaux ne doivent pas contenir de mots de passe, de jetons d'accès ni de contenus documentaires sensibles.

3.3.8 Amazon SES

SES peut envoyer un e-mail au créateur lorsqu'un lien atteint son dernier téléchargement autorisé.

L'événement métier doit être déclenché de manière fiable et idempotente afin d'éviter les notifications multiples.

La configuration d'envoi, les identités vérifiées et les restrictions du compte SES doivent être contrôlées avant la mise en production.

Chapitre 4

Conception de la collaboration temps réel

4.1 Problème de la concurrence

Plusieurs utilisateurs peuvent modifier le même document simultanément. Une stratégie naïve consistant à remplacer le contenu complet par la dernière requête reçue peut entraîner la perte des modifications précédentes.

La solution doit distinguer :

- l'état local de chaque éditeur ;
- les modifications produites par cet éditeur ;
- les modifications reçues des autres participants ;
- l'état persistant du document ;
- les versions historiques destinées à la restauration.

4.2 Utilisation de Yjs

Yjs est envisagé comme moteur de collaboration basé sur des structures de données répliquées sans conflit, appelées CRDT.

Les clients produisent des mises à jour qui peuvent être fusionnées. Le serveur distribue ces mises à jour aux autres participants et assure leur persistance.

Le serveur doit contrôler les permissions avant d'accepter une mise à jour. Une bibliothèque CRDT ne remplace ni l'authentification ni l'autorisation.

4.3 Protocole de synchronisation

Un message de synchronisation comprend au minimum :

- un identifiant de document ;
- un type de message ;
- une charge utile correspondant au protocole choisi ;
- un identifiant de corrélation si nécessaire ;
- des informations de version ou d'accusé de réception adaptées au mécanisme de persistance.

La charge utile doit être validée. La taille maximale des messages doit être limitée pour éviter les abus et les consommations excessives de mémoire.

4.4 Présence et curseurs

La présence contient des informations temporaires : utilisateurs connectés, curseurs, sélections et activité.

Ces données peuvent être diffusées sans créer une nouvelle version documentaire à chaque déplacement de curseur.

La présence doit expirer après la fermeture ou la perte d'une connexion. Les connexions devenues invalides doivent être retirées de l'état de présence.

4.5 Persistance et reconnexion

Le serveur persiste les mises à jour ou des états documentaires compacts, puis permet aux clients de récupérer l'état nécessaire à leur reconnexion.

Les opérations de compactage doivent être conçues pour éviter la suppression d'une mise à jour qui n'a pas encore été durablement enregistrée.

Le format exact de persistance dépendra de l'implémentation retenue : mises à jour binaires, snapshots ou combinaison des deux.

4.6 Versions et restauration

Une version historique représente un état cohérent du document à un instant déterminé.

La restauration d'une version ne doit pas écraser silencieusement l'historique. Elle crée une nouvelle version courante dérivée du contenu restauré.

Chaque version comporte un identifiant, une référence au document, un numéro ou identifiant ordonné, un horodatage, un auteur ou acteur et une référence au contenu persisté.

Chapitre 5

Modèle de données

5.1 Principes

Le modèle sépare les métadonnées fréquemment consultées des contenus documentaires et des fichiers PDF.

Les identifiants internes sont générés côté serveur. Les jetons de partage et de téléchargement sont aléatoires, non prédictibles et ne sont pas enregistrés en clair lorsqu'une empreinte suffit.

Les structures ci-dessous sont logiques : elles devront être traduites en clés primaires, clés de tri et index secondaires selon les accès réellement nécessaires.

5.2 Entités principales

Entité	**Attributs principaux**
User	userId, email vérifié si disponible, displayName, createdAt, status.
Document	documentId, ownerId, title, currentVersionId, createdAt, updatedAt, status.
DocumentPermission	documentId, principalId, role, grantedAt, grantedBy.
DocumentVersion	documentId, versionId, parentVersionId, contentReference, createdAt, createdBy, checksum.
ShareLink	linkId, documentId, permissions, expiresAt, status, tokenHash.
DownloadCounter	linkId, remainingCount, totalAuthorized, updatedAt.
DailyQuota	principalId, quotaDate, charactersUsed, lastUpdatedAt.
AbuseBlock	principalId, reasonCode, blockedUntil, createdAt.
Association	associationId, name, status, createdAt.
AssociationMember	associationId, principalId, role, joinedAt.
NotificationEvent	eventId, type, recipient, status, createdAt, attemptCount.
AuditEvent	eventId, actorId, action, resourceId, timestamp, resultCode.

5.3 Clés et index

La clé primaire de chaque table ou item doit être choisie selon les requêtes nécessaires. Les règles suivantes guident la conception :

— les documents sont consultables par leur identifiant ;
— les versions sont consultables par document et ordre de version ;
— les quotas sont isolés par rédacteur et journée ;
— le compteur de téléchargement est associé à un lien ;
— les permissions permettent de vérifier l'accès à un document sans parcourir toute la base ;
— les événements de notification sont identifiables de manière idempotente.

Un index secondaire ne doit être ajouté que si un accès identifié le justifie. Le choix des clés doit aussi tenir compte de la répartition de charge et des risques de partitions chaudes.

5.4 Persistance des contenus

Les métadonnées sont stockées dans DynamoDB. Les contenus volumineux et les PDF peuvent être stockés dans S3.

Chaque référence S3 doit désigner un objet précis. Un PDF généré pour un téléchargement doit être lié à un identifiant de version et à une empreinte de contenu.

Le document courant ne doit pas être lu à plusieurs moments sans contrôle de version lors d'une génération : cela pourrait produire un export mélangeant des états différents.

Chapitre 6

Gestion des quotas et téléchargements

6.1 Quota quotidien de rédaction

Le quota est fixé à 10 000 caractères par rédacteur et par jour.

La définition du caractère doit être stable. Par exemple, le système peut compter les points de code Unicode après normalisation du texte, et non le nombre d'octets UTF-8. Cette convention doit être appliquée de manière identique dans tous les chemins d'écriture.

Le jour de quota est défini selon un fuseau horaire explicite. Pour éviter toute ambiguïté, le système peut utiliser UTC.

6.1.1 Contrôle atomique

Une opération de modification doit vérifier que le quota restant est suffisant avant de valider l'augmentation de consommation.

Une mise à jour conditionnelle peut prendre la forme logique :

```
charactersUsed + requestedCharacters <= 10000
```

Listing 6.1 – Condition métier du quota

Cette expression illustre la règle métier et non une requête SQL exécutable sur DynamoDB. Dans DynamoDB, le contrôle doit être réalisé au moyen d'une expression conditionnelle et d'une mise à jour atomique, ou d'une transaction si plusieurs éléments doivent être modifiés ensemble.

La stratégie de comptage doit également définir comment traiter les suppressions, les annulations, les retransmissions et les requêtes répétées. Un identifiant d'opération peut être utilisé pour rendre les écritures idempotentes.

6.2 Création d'un lien de partage

Un lien contient un jeton aléatoire suffisamment robuste. Le serveur conserve l'empreinte du jeton plutôt que le secret en clair, lorsque cela est applicable.

Le lien peut comporter :

— une référence au document ;
— les permissions accordées ;

— une date d'expiration ;
— un état actif ou révoqué ;
— un quota maximal de téléchargements ;
— un compteur des téléchargements autorisés.

Le jeton doit être protégé contre les fuites dans les journaux, les référents HTTP et les outils d'analyse.

6.3 Contrôle concurrent des téléchargements

Le contrôle du nombre de téléchargements doit être réalisé côté serveur. Une vérification suivie d'une décrémentation non atomique permettrait à plusieurs requêtes simultanées de dépasser la limite.

Le serveur utilise une écriture conditionnelle atomique ou une transaction DynamoDB pour réserver un téléchargement seulement si :

— le lien existe ;
— le lien est actif et non expiré ;
— le compteur restant est strictement positif ;
— le document et le fichier ciblés sont valides.

La décrémentation ne doit être effectuée qu'une seule fois par opération logique, y compris lors des tentatives répétées.

6.4 Réservation et livraison effective

Il faut distinguer trois événements :

1. la réservation atomique d'un droit de téléchargement ;
2. le démarrage de la transmission du fichier ;
3. la confirmation que le client a reçu le fichier.

Le serveur peut prouver les deux premiers événements selon son protocole et ses journaux. En revanche, une réponse HTTP réussie ne garantit pas toujours que l'utilisateur a enregistré le fichier complet sur son appareil.

Par conséquent, le sens de la limite doit être défini explicitement. Dans la première version, le compteur représente le nombre de téléchargements autorisés et réservés côté serveur. Une garantie de livraison physique exacte nécessiterait un protocole de livraison et de confirmation supplémentaire, sans pouvoir éliminer toutes les ambiguïtés dues aux coupures réseau.

6.5 Livraison des PDF

Les PDF sont conservés dans un bucket S3 privé.

Pour éviter qu'une URL présignée réutilisable contourne le compteur, la livraison doit passer par un mécanisme contrôlé par le backend. Une URL présignée S3 standard ne constitue pas, à elle seule, un jeton à usage unique.

La première version peut imposer une taille maximale de PDF compatible avec les limites de réponse de l'architecture retenue. Si les fichiers sont plus volumineux, un mécanisme de livraison spécialisé devra être évalué avant la mise en production.

La version du document utilisée pour produire le PDF est figée au début de la génération. Le fichier est associé à cette version, et son empreinte permet d'en vérifier l'intégrité.

6.6 Notification du dernier téléchargement

Lorsque le compteur passe de 1 à 0, le serveur produit un événement de notification.

Cet événement doit être créé de manière fiable avec l'opération métier concernée, par exemple via une transaction ou un mécanisme d'outbox. Une notification déjà envoyée ne doit pas être réémise à chaque nouvelle tentative de traitement.

La livraison de l'e-mail est asynchrone. Les erreurs d'envoi doivent être journalisées et les tentatives contrôlées.

Chapitre 7

Sécurité

7.1 Gestion des identités et des permissions

Le principe du moindre privilège est appliqué à toutes les ressources AWS.

Chaque fonction Lambda reçoit un rôle IAM limité aux actions et ressources nécessaires. Une fonction qui lit des métadonnées ne reçoit pas automatiquement des droits d'écriture sur S3.

Les permissions applicatives sont vérifiées côté serveur. Les rôles applicatifs, tels que propriétaire, rédacteur et lecteur, sont distincts des rôles IAM utilisés par AWS.

Si l'application utilise une identité déclarative ou une adresse e-mail, ces éléments ne doivent pas être considérés comme une preuve d'identité sans vérification adaptée.

7.2 Protection des données

Les mesures suivantes sont prévues :

- bucket S3 privé et blocage de l'accès public ;
- chiffrement au repos avec les mécanismes appropriés ;
- HTTPS pour les communications externes ;
- validation des entrées et contrôle des tailles ;
- protection des jetons de partage ;
- absence de secrets dans le code source ;
- limitation des informations personnelles conservées ;
- règles de rétention et suppression documentées ;
- journalisation sans contenu sensible.

7.3 Protection contre les abus

Le système peut limiter :

- les requêtes par identité ou par adresse IP ;
- le nombre de connexions simultanées ;
- la fréquence des modifications ;
- la taille des messages WebSocket ;
- la fréquence de création des liens ;
- les tentatives répétées de téléchargement.

Les seuils doivent être configurables et testés. Les mécanismes de limitation doivent éviter de bloquer injustement des utilisateurs légitimes derrière une même adresse IP.

Un blocage temporaire possède une date d'expiration et un motif codifié. Les informations utilisées pour la détection doivent être limitées au nécessaire.

7.4 Sécurité réseau

Les API publiques doivent être exposées uniquement via les points d'entrée nécessaires. Les ressources de stockage ne doivent pas être rendues publiques pour simplifier les accès.

L'utilisation d'un VPC pour certaines fonctions Lambda sera décidée en fonction des besoins réseau réels. Un VPC n'est pas automatiquement requis pour chaque fonction Lambda.

Les groupes de sécurité et les sous-réseaux doivent être configurés selon le principe d'exposition minimale lorsqu'un VPC est utilisé.

7.5 Gestion des secrets

Les identifiants AWS ne doivent jamais être intégrés au dépôt Git ni au bundle frontend.

Les accès de développement reposent sur des mécanismes d'authentification appropriés. Les secrets applicatifs éventuels sont stockés dans un service adapté et leur accès est limité.

Le dépôt doit exclure les fichiers de configuration locaux contenant des secrets, les artefacts de compilation et les fichiers temporaires.

Chapitre 8

Scalabilité et disponibilité

8.1 Hypothèse de charge

Le dimensionnement initial considère le scénario suivant :

— 15 000 visiteurs sur une période de forte affluence ;
— 80 % de ces visiteurs arrivent pendant la première minute ;
— 300 rédacteurs peuvent poursuivre leurs modifications pendant cette période.

Le nombre de visiteurs arrivant pendant la première minute est :

$$15\,000 \times 0{,}80 = 12\,000$$

Le débit moyen d'arrivée correspondant est :

$$\frac{12\,000}{60} = 200 \quad \text{visiteurs par seconde}$$

Cette valeur représente un débit moyen d'arrivée de visiteurs, et non automatiquement 200 requêtes HTTP par seconde. Le débit réel dépendra du nombre de requêtes effectuées par visiteur, du nombre de connexions WebSocket et du comportement de l'interface.

8.2 Stratégie de montée en charge

8.2.1 Lambda

La concurrence des fonctions doit être surveillée. Les quotas de concurrence du compte et les éventuelles limites réservées doivent être vérifiés avant les tests.

Le code doit limiter les appels redondants et les traitements synchrones coûteux. Les délais d'expiration et les reprises doivent être définis selon la nature des opérations.

8.2.2 API Gateway

Les limites de débit et les quotas d'API Gateway doivent être vérifiés dans la région retenue.

Les connexions WebSocket et les messages échangés sont dimensionnés à partir d'un modèle de trafic mesuré. Les reconnexions massives doivent être testées.

8.2.3 DynamoDB

Le mode de capacité, à la demande ou provisionnée, doit être choisi en fonction des mesures et des coûts.

Il faut surveiller la latence, les erreurs de limitation, la distribution des clés et les accès fortement concentrés sur un même document.

Les compteurs globaux très sollicités peuvent devenir un point de contention. Leur conception doit préserver l'exactitude des règles métier avant toute optimisation.

8.2.4 S3

S3 stocke les fichiers indépendamment de l'exécution des fonctions Lambda. Les téléchargements ne doivent cependant pas contourner les contrôles d'autorisation ou de quota.

Les tailles des fichiers, les fréquences de lecture et les coûts de transfert doivent être mesurés.

8.3 Disponibilité et reprise

La disponibilité repose sur les capacités des services managés, mais ne doit pas être considérée comme automatique.

Le système doit prévoir :

- les erreurs temporaires des services ;
- les timeouts ;
- les reconnexions WebSocket ;
- les reprises idempotentes ;
- la sauvegarde et la restauration des données ;
- la vérification de l'intégrité des versions ;
- la surveillance des quotas AWS.

La stratégie de sauvegarde, la durée de rétention et les objectifs de reprise doivent être définis en fonction des besoins métier.

Chapitre 9

Infrastructure as Code et déploiement

9.1 Rôle d'AWS SAM

AWS SAM permet de déclarer les ressources serverless dans un template. Il facilite la validation, la compilation et le déploiement de l'application.

SAM s'appuie sur CloudFormation pour gérer les ressources AWS. Le template est versionné avec le code afin de maintenir une traçabilité des changements d'infrastructure.

9.2 Organisation du dépôt

L'organisation cible est la suivante :

```
shared-docs-editor/
|-- frontend/
|   |-- src/
|   |-- public/
|   |-- package.json
|   `-- vite.config.ts
|-- backend/
|   |-- src/
|   |-- tests/
|   |-- package.json
|   `-- package-lock.json
|-- infrastructure/
|   `-- template.yaml
|-- docs/
|   |-- conception.tex
|   |-- api.md
|   `-- rapport-tests.md
|-- scripts/
|-- .gitignore
`-- README.md
```

Listing 9.1 – Structure indicative du dépôt

Cette structure est une cible d'organisation. Elle doit être adaptée à l'état réel du dépôt et aux fichiers effectivement présents.

9.3 Étapes de validation

Avant un déploiement, les contrôles suivants sont réalisés :

1. vérifier la structure et les dépendances du dépôt ;
2. installer les dépendances de manière reproductible ;
3. exécuter le contrôle de types ;
4. exécuter les tests unitaires ;
5. compiler le frontend ;
6. valider le template SAM ;
7. construire le backend avec SAM ;
8. examiner les permissions IAM et les changements prévus ;
9. vérifier la configuration de la région et du compte ;
10. déployer uniquement après validation explicite.

9.4 Commandes indicatives

Les commandes suivantes illustrent le processus attendu. Les chemins doivent être adaptés au dépôt réel.

```
# Depuis la racine du projet
sam validate --template-file infrastructure/template.yaml --lint

sam build --template-file infrastructure/template.yaml
```

Listing 9.2 – Validation et construction

Le déploiement est une étape distincte de la compilation. Il ne doit être lancé qu'après vérification du compte AWS, de la région, des permissions, des ressources et des coûts.

9.5 Suppression des ressources

La suppression d'une stack CloudFormation peut supprimer les ressources qu'elle gère. Certaines ressources peuvent être conservées selon leurs politiques de suppression ou parce qu'elles ont été créées séparément.

Avant toute suppression :

— identifier le nom exact de la stack ;
— vérifier le compte et la région ;
— examiner les ressources et leurs politiques ;
— identifier les données à conserver ;
— prévoir la gestion des buckets non vides ;
— vérifier les ressources indépendantes de la stack.

La suppression ne doit jamais être lancée sur la seule base d'un nom supposé.

Chapitre 10

Plan de tests

10.1 Objectifs

Les tests doivent démontrer le fonctionnement des exigences fonctionnelles, la cohérence des données, la sécurité et le comportement sous charge.

Les résultats doivent être reproductibles et accompagnés de données brutes, d'une méthode de mesure et d'une analyse.

Aucun résultat ne doit être présenté comme mesuré avant l'exécution effective du test.

10.2 Tests unitaires

Les tests unitaires couvrent notamment :

- la validation des requêtes ;
- le calcul du quota quotidien ;
- la validation des liens de partage ;
- les permissions sur les documents ;
- la création de versions ;
- la logique de restauration ;
- la génération des événements de notification ;
- la gestion des erreurs.

10.3 Tests d'intégration

Les tests d'intégration vérifient les interactions entre :

- API Gateway et les fonctions Lambda ;
- Lambda et DynamoDB ;
- Lambda et S3 ;
- le transport WebSocket et la synchronisation Yjs ;
- les opérations métier et le système de notification.

10.4 Tests de concurrence

Les scénarios critiques comprennent :

1. deux rédacteurs modifient le même paragraphe ;
2. plusieurs mises à jour sont reçues dans un ordre différent ;
3. un client se déconnecte puis se reconnecte ;
4. plusieurs requêtes tentent de consommer simultanément le dernier téléchargement disponible ;
5. plusieurs modifications tentent de consommer le quota restant d'un même rédacteur ;
6. une notification est traitée plusieurs fois.

Les invariants suivants doivent être vérifiés :

— le compteur de téléchargement ne devient jamais négatif ;
— le nombre de réservations acceptées ne dépasse pas le nombre maximal autorisé ;
— les écritures concurrentes respectent le quota ;
— les opérations répétées ne produisent pas de double effet métier lorsque l'idempotence est requise ;
— les modifications valides convergent vers un état cohérent ;
— les versions restaurées restent identifiables dans l'historique.

10.5 Tests de charge

Le plan de charge doit distinguer les arrivées de visiteurs, les requêtes HTTP et les connexions WebSocket.

Les scénarios comprennent :

— une charge progressive ;
— une arrivée de 12 000 visiteurs sur 60 secondes, selon un profil d'arrivée documenté ;
— 300 rédacteurs actifs pendant le pic ;
— des modifications concurrentes ;
— des lectures de documents ;
— des demandes de téléchargement ;
— des déconnexions et reconnexions.

Le test doit respecter les quotas AWS et ne doit pas être lancé à grande échelle sans estimation préalable des coûts.

10.6 Métriques à recueillir

Métrique	**Utilité**
Nombre d'arrivées	Vérifier le profil de charge généré.
Requêtes par seconde	Mesurer le débit réel des API.
Latence p50, p95, p99	Caractériser les temps de réponse.
Taux d'erreurs	Identifier les échecs HTTP et applicatifs.
Durée d'exécution Lambda	Détecter les traitements coûteux.
Concurrence Lambda	Identifier les risques de saturation.
Latence et erreurs DynamoDB	Détecter les limitations et les accès critiques.

Connexions WebSocket	Mesurer la charge de collaboration.
Temps de synchronisation	Mesurer le délai entre une modification et sa réception par les autres clients.
Compteurs de téléchargement	Vérifier le respect des limites.
Consommation du quota	Vérifier les règles de rédaction.
Coût estimé	Relier le scénario de test à la consommation AWS.

10.7 Rapport de test

Chaque test doit comporter :

- un identifiant et un objectif ;
- les préconditions et la configuration ;
- la version du code et de l'infrastructure ;
- le profil de charge et les paramètres ;
- les données brutes et les résultats ;
- les erreurs et les anomalies observées ;
- une analyse des résultats ;
- une conclusion et les limites du test.

Chapitre 11

Documentation API

11.1 Principes

L'API est documentée avec les schémas de requêtes et de réponses, les codes HTTP, les règles d'autorisation, les erreurs et les limites de débit.

Les routes ci-dessous sont indicatives. Elles représentent l'interface cible et ne garantissent pas que ces routes sont déjà implémentées.

11.2 Routes HTTP envisagées

Méthode	Route indicative	Fonction
POST	/documents	Créer un document.
GET	/documents/{id}	Lire les métadonnées et l'état autorisé.
PATCH	/documents/{id}	Modifier les métadonnées autorisées.
GET	/documents/{id}/versions	Lister les versions.
POST	/documents/{id}/restore	Restaurer une version.
POST	/documents/{id}/share-links	Créer un lien de partage.
POST	/documents/{id}/exports	Demander la génération d'un PDF.
POST	/download-links/{token}/download	Réserver et démarrer un téléchargement contrôlé.
GET	/associations/{id}/dashboard	Consulter le tableau de bord autorisé.

11.3 Codes de réponse

Les codes suivants peuvent être utilisés selon le contexte :

— `200 OK` : opération de lecture réussie ;
— `201 Created` : ressource créée ;
— `400 Bad Request` : requête invalide ;
— `401 Unauthorized` : identité non authentifiée lorsque l'authentification est requise ;
— `403 Forbidden` : permission insuffisante ;

— `404 Not Found` : ressource absente ou non exposée ;
— `409 Conflict` : conflit métier ou version ;
— `410 Gone` : lien expiré ou épuisé, si cette sémantique est retenue ;
— `429 Too Many Requests` : limitation de débit ;
— `500 Internal Server Error` : erreur inattendue ;
— `503 Service Unavailable` : indisponibilité temporaire.

Les réponses ne doivent pas divulguer de détails internes, de secrets ni de traces d'exécution.

Chapitre 12

Coûts et maîtrise budgétaire

12.1 Principaux postes de coût

Le coût dépend de la région, du nombre de requêtes, des durées d'exécution, du stockage, des transferts et du mode de capacité.

Les postes à examiner sont :

- exécutions et concurrence Lambda ;
- requêtes HTTP et connexions/messages API Gateway ;
- lectures, écritures et stockage DynamoDB ;
- stockage, requêtes et transferts S3 ;
- journaux et métriques CloudWatch ;
- envois d'e-mails SES ;
- transferts de données sortants ;
- outils de test et services supplémentaires éventuels.

12.2 Méthode d'estimation

L'estimation budgétaire doit partir de scénarios mesurables : nombre de visiteurs, nombre moyen de requêtes par visiteur, durée moyenne des fonctions, volume de données stockées, taille des PDF et nombre de téléchargements.

Une estimation doit indiquer la région, la période, les hypothèses et les tarifs consultés.

Les tarifs ne sont pas intégrés ici sous forme de montants fixes, car ils peuvent évoluer et dépendent de la consommation réelle.

12.3 Alertes et limites

Des budgets AWS peuvent déclencher des alertes à différents seuils, par exemple 50 %, 80 % et 100 % du budget.

Ces alertes servent à informer l'équipe ; elles ne constituent pas un plafond de dépenses garanti et n'arrêtent pas automatiquement toutes les ressources.

La prévention des coûts repose également sur :

- des tests de charge progressifs ;
- la suppression des ressources de test inutilisées ;

— la vérification des ressources conservées ;
— la rétention maîtrisée des journaux ;
— le suivi des coûts après chaque test.

Chapitre 13

Risques et mesures d'atténuation

Risque	Conséquence	Mesure
Perte de modifications	Incohérence documentaire	CRDT, persistance contrôlée, tests de reconnexion.
Dépassement des quotas	Violation des règles métier	Écritures conditionnelles et tests de concurrence.
Lien réutilisable	Téléchargements supplémentaires	Contrôle côté serveur et absence de contournement par URL présignée réutilisable.
Pic de trafic	Latence ou erreurs	Mesures, vérification des quotas AWS et tests progressifs.
Permissions excessives	Accès non autorisé	Rôles IAM dédiés et principe du moindre privilège.
Coût inattendu	Dépassement budgétaire	Estimations, alertes, mesures et nettoyage contrôlé.
Notification répétée	Spam ou confusion	Événements idempotents et suivi de l'état d'envoi.
Mauvaise configuration	Exposition ou indisponibilité	Validation IaC, revue des changements et tests.

Chapitre 14

Plan de réalisation

La réalisation est organisée en étapes successives.

Phase	Objectif	Livrables
1	Analyse et conception	Exigences validées, architecture, modèle de données et décisions techniques.
2	Socle technique	Dépôt organisé, dépendances, configuration, tests et template SAM validé.
3	Documents	Création, lecture, modification et persistance.
4	Collaboration	Synchronisation, présence, reconnexion et persistance des mises à jour.
5	Versions	Historique, restauration et contrôle d'intégrité.
6	Partage et PDF	Liens, génération de PDF, quotas et téléchargements contrôlés.
7	Sécurité	Permissions, validation, blocages et audit.
8	Supervision	Journaux, métriques et alertes.
9	Tests	Tests fonctionnels, concurrence, charge et sécurité.
10	Livraison	Documentation, résultats, coûts et procédures de déploiement et suppression.

14.1 Critères de validation

Le projet est considéré comme validé lorsque :

- les fonctionnalités du périmètre sont testées ;
- les modifications concurrentes restent cohérentes ;
- les quotas sont respectés sous concurrence ;
- les liens épuisés ne permettent plus de réserver de téléchargement ;
- les PDF sont associés à une version cohérente ;
- le stockage n'est pas publiquement accessible ;
- les rôles IAM sont limités et documentés ;
- le build et le déploiement sont reproductibles ;
- les résultats des tests sont accompagnés de preuves ;

— les coûts et les limites de la solution sont documentés.

Chapitre 15

Conclusion

Shared Documents Editor vise à combiner collaboration en temps réel, persistance documentaire, partage contrôlé et infrastructure cloud reproductible.

L'architecture AWS retenue répartit les responsabilités entre les API, les fonctions de traitement, la base de données, le stockage objet et la supervision.

L'utilisation d'AWS SAM et de CloudFormation permet de décrire l'infrastructure comme du code. Le recours à IAM, aux buckets privés, aux écritures conditionnelles et à la journalisation contribue à appliquer les bonnes pratiques de sécurité, de cohérence et d'exploitation.

La validation définitive dépendra toutefois des tests réels. Les performances, les quotas de service, les coûts et la garantie exacte du comportement des téléchargements ne doivent pas être considérés comme acquis avant vérification.

Le dossier de conception sera complété par la documentation des API, le rapport des tests exécutés, les mesures brutes, les résultats de charge et les preuves de configuration des ressources effectivement déployées.